

# Aquafoundry Social Media Generation Workflow

Complete setup guide, node reference, prompt library, Google Sheets schema, and improvement roadmap for the n8n AI content automation pipeline.

| WORKFLOW FILE            | PLATFORM     | TOTAL NODES | IMAGE STORAGE | OUTPUT DATA   |
|--------------------------|--------------|-------------|---------------|---------------|
| social-media-gen-v8.json | n8n + Gemini | 39 Nodes    | Supabase CDN  | Google Sheets |

## 1. Overview

This n8n workflow automates the entire social media content pipeline for Aquafoundry—an Indian aquaponics brand. Starting from a simple form submission, the system plans a multi-day Instagram content calendar, generates product-aware AI images, overlays brand assets, applies real product labels, and stores the final ad images in Supabase with public URLs written back to Google Sheets.

### Pipeline Operational Note

The workflow has **three distinct pipelines** that run independently. You must run Phase 1 (Asset Catalog) once before running Phases 2 and 3, which then work together to produce ready-to-post Instagram ads.

### What the Workflow Produces

- A structured **content plan** in Google Sheets with captions, headlines, CTAs, image prompts, and post metadata for every day of your campaign.
- **AI-generated raw images** for every Post (not Reel)—product-focused, produce-focused, or lifestyle scenes.
- **Final polished Instagram ads** with the brand's logo, mascot character, and real product labels composited in.
- **Public image URLs** stored in Supabase and written back to the content plan sheet.

Core Technical Stack

| Component           | Service / Tool                                                 | Purpose                                     |
|---------------------|----------------------------------------------------------------|---------------------------------------------|
| Orchestration       | n8n (self-hosted or cloud)                                     | Workflow automation engine                  |
| AI Text             | Google Gemini Flash (gemini-flash-latest)                      | Content strategy generation, image analysis |
| AI Image Generation | Google Gemini 3.1 Flash Image (gemini-3.1-flash-image-preview) | Raw ad image creation and final composition |
| Content Database    | Google Sheets                                                  | Content plan + Asset catalog storage        |
| Calendar Context    | Google Calendar (India Holidays)                               | Holiday-aware content planning              |
| Image Storage       | Supabase Storage                                               | Final ad image CDN + public URLs            |
| Form Input          | n8n Form Trigger                                               | Campaign date entry UI                      |

2. System Architecture

The 39 nodes are organized into three independent pipelines. Each pipeline has a distinct trigger and structural purpose:

PHASE 1 — ASSET CATALOG INITIALIZATION (RUN ONCE)

Scans all product images from Supabase, uses Gemini Vision to write detailed product descriptions, and populates the `Asset_Catalog` sheet. This must complete before running Phase 2.

PHASE 2 — CONTENT STRATEGY PLANNING (FORM-TRIGGERED)

Accepts campaign start/end dates via a web form, fetches Indian holidays, reads the asset catalog, sends a large context prompt to Gemini, and writes a structured multi-day content plan to `Sheet1`.

PHASE 3 — IMAGE GENERATION LOOP (MANUALLY TRIGGERED)

Reads all pending "Post" rows from `Sheet1`, loops through each, generates a raw AI image, analyzes it to pick the best logo and mascot, downloads brand assets, applies product labels, composites the final ad, and uploads to Supabase.

Data Flow Text Representation

```
PHASE 1 (One-time):
Build Product Image List -> Get Product Image -> Gemini Analyze Image
  -> Parse Product Description -> Append to Asset_Catalog Sheet

PHASE 2 (Per Campaign):
Form (dates) -> Parse Dates -> Get Indian Holidays -> Summarize
  -> Get Product Asset Catalog -> Build Brand Context
```

```
-> Gemini Content Strategy -> Parse & Enrich JSON -> Append to Sheet1
```

PHASE 3 (Per Post Loop):

Manual Trigger -> Get Pending Rows -> Filter Posts -> Loop Over Batches

```
-> Build Runtime Asset Catalog -> Gemini Generate Raw Image
```

```
-> Preserve Raw Image -> Gemini Analyze (pick logo + mascot)
```

```
-> Download Assets (Mascot & Logo) -> Is Packaging Locked?
```

```
  |-- YES -> Get Product Label -> Edit Packaging Locked Image
```

```
  |-- NO -> Edit Creative Food Scene Image
```

```
-> Normalize Binary -> Upload to Supabase -> Update Sheet1 Row -> Loop End
```

## 3. Prerequisites & Setup

Before importing and running the workflow, verify all of the following configurations are complete:

### 3.1 Google Sheets Workbook Schema

The workflow uses a single Google Sheets file with **two tabs**. The default Spreadsheet ID used is . When setting up your own copy, ensure this ID is updated in every Google Sheets node.

#### SHEET1 — CONTENT PLAN (TAB 1)

Stores the full content plan generated by Phase 2, plus the image URLs written back by Phase 3.

| Column Name    | Type   | Set By    | Description                                                                     |
|----------------|--------|-----------|---------------------------------------------------------------------------------|
| Row_ID         | String | Phase 2   | Unique ID for each row (e.g., "Day-1-Post-1"). Used as update key.              |
| Day            | String | Phase 2   | e.g., "Day 1: 2026-07-01 - Fresh Herb Season"                                   |
| Post_Type      | String | Phase 2   | "Post" or "Reel"—Phase 3 only processes "Post".                                 |
| Image_Prompt   | String | Phase 2   | The full AI image generation prompt from Gemini strategy.                       |
| Caption        | String | Phase 2   | Instagram caption with relevant campaign hashtags.                              |
| Headline       | String | Phase 2   | 3–6 word ad headline overlay text.                                              |
| Status         | String | Phase 2/3 | <b>pending</b> (set by Phase 2) or <b>completed</b> (set by Phase 3).           |
| Rendering_Mode | String | Phase 2   | <b>packaging_locked</b> (jars/bottles) or <b>creative_food_scene</b> (produce). |
| Image_URL      | String | Phase 3   | Public Supabase CDN URL of the final generated ad image.                        |

#### ASSET\_CATALOG SHEET (TAB 2)

Populated by Phase 1. Each row represents one product with AI-generated visual descriptions.

| Column Name        | Source Phase | Description                                                     |
|--------------------|--------------|-----------------------------------------------------------------|
| Asset_ID           | Phase 1      | Unique product identifier (e.g., "basil_pesto", "rocket_pesto") |
| Asset_URL          | Phase 1      | Supabase URL of the product baseline reference image            |
| Visual_Description | Phase 1 (AI) | 2–4 sentence structural visual description by Gemini Vision     |
| Rendering_Mode     | Phase 1 (JS) | Governs whether a physical product label must be applied        |
| Label_URL          | Phase 1 (JS) | Supabase URL to the exact transparent product label PNG         |

### 3.2 Supabase Storage Bucket Structure

The workflow utilizes a public Supabase Storage bucket named `aquaponics-media`. Ensure your asset files strictly adhere to the path structures below:

```

aquaponics-media/
├── assets/
│   ├── products/           (e.g., product.png, product1.png for baseline imagery)
│   ├── labels/             (e.g., basil-pesto.png transparent asset wrappers)
│   ├── logos/              (e.g., logo1.png for light, logo2.png for dark backgrounds)
│   └── mascots/            (e.g., mascot1.png, mascot2.png character states)
└── [Final generated ad output files are written to the root directory automatically]
```

## 4. Operational Phase Deep-Dive

### Phase 1 — Asset Catalog Initialization

This is an execution chain triggered manually to index creative assets prior to launching a campaign strategy script.

- Build Product Image List (JS Code):** Initializes the structural array containing reference parameters for the 8 baseline product types.
- Get Product Image (HTTP Request):** Streams binary data from the remote asset bucket into local memory storage context.
- Gemini Analyze Product Image:** Evaluates geometric visual rules, material traits, and context using `gemini-2.5-flash`.
- Parse Product Description:** Sanitizes output blocks and formats parameters into database-safe objects.
- Append Row:** Commits data to the `Asset_Catalog` sheet.

### Phase 2 — Content Strategy Planning

Triggered dynamically by a consumer-facing web interface when a specific campaign calendar bracket is defined.

- Form Trigger:** Captures calendar start and stop boundaries securely.

2. **Parse Dates:** Transforms criteria into specific timestamp integers and intervals.
3. **Get Indian Holidays:** Queries regional calendars to optimize structural alignment with domestic public events.
4. **Gemini Content Strategy Execution:** Leverages a highly extensive technical prompt template with `gemini-flash-latest` to map individual content records (1 Post + 1 Reel per day) into a structured JSON configuration array.
5. **Append Content Matrix:** Serializes planned days directly into rows inside `Sheet1`.

## Phase 3 — Image Generation Loop

Processes structural data arrays row-by-row utilizing specialized batch handlers and dynamic logic loops.

1. **Filter Rows:** Isolates targets meeting criteria where `Post_Type === "Post"` and status is labeled `pending`.
2. **Gemini Generation Core:** Creates contextual graphics using `gemini-3.1-flash-image-preview` strictly mapping to baseline color guides without branding text.
3. **Asset Optimization Selection:** Feeds generated frames into an analysis stage where specific light/dark logos and character assets are dynamically chosen based on ambient space metrics.
4. **Compositing Execution:** Forks into separate branches based on rendering definitions. Jars undergo structural perspective wrapping to merge physical product label files onto background layers smoothly. Fresh scenes default directly to typographical balance layers.
5. **Storage Commit:** Pushes finalized objects to the production CDN and marks records as `completed`.

## 5. Complete Node Reference

The following details key functional nodes across all 39 workflow configurations:

### Get row(s) in sheet1

`n8n-nodes-base.googleSheets`

Reads all raw structural content metadata across rows from the target workspace. Connected directly to manual workflow execution hooks.

### If1 Filter Handler

`n8n-nodes-base.if`

Validates processing eligibility metrics. Enforces strict logical filtering matching targets where type equals Post and status matches pending.

### Generate an image

`@n8n/nodes-langchain.googleGemini`

Leverages `gemini-3.1-flash-image-preview` to map textual prompts into unbranded commercial background plates.

### SUPAbase\_Image\_storage

`n8n-nodes-base.httpRequest`

Pushes active media payloads directly to target web buckets using secure authorization tokens.

## 6. Prompt Library Extract



**Node:** Gemini Content Strategy (Phase 2) | **Model:** gemini-flash-latest

Act as a Senior Social Media Creative Director and AI Advertising Strategist.  
Create a high-engagement content strategy for Aquafoundry for specified campaign duration days.  
The goal is NOT to create generic aquaponics content.  
The goal is to create a premium product-aware Instagram content plan where:

- some posts focus on Aquafoundry products
- some posts focus on fresh produce outcomes
- some posts focus on Indian urban lifestyle

Ensure strict output schemas formatted as a valid JSON array matching the parameters provided.

 **Node: Edit Image Packaging Locked (Phase 3) | Model: gemini-3.1-flash-image-preview**

Transform this raw Aquafoundry image into a premium Instagram ad creative.  
Use: uploaded raw image + uploaded logo + uploaded mascot + uploaded product\_label  
PRODUCT LABEL LOCK:  
The uploaded product\_label is the ONLY source of truth. Do NOT redesign it or alter text.  
Apply the uploaded product\_label naturally onto the blank packaging area using high precision perspective fit and realistic lighting adaptation.

## 7. Troubleshooting & Roadmap Matrix

| Error State                        | Root Trigger Condition                                     | Required Remediation Mitigation                                                |
|------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------|
| No JSON array found in AI response | Context limit exceeded or raw markdown wrapper leakage.    | Reduce operational planning window lengths down into smaller 7-day batches.    |
| Supabase 401 Error                 | Expired structural authorization bearer key pairs.         | Regenerate service roles key vectors and map to secure runtime containers.     |
| Distorted Brand Mascot Elements    | Generative leakage during rendering refinement operations. | Strengthen structural asset preservation statements inside prompt definitions. |

### Strategic High-Priority Improvements

- **Dynamic Configuration Tables:** Transition from static code blocks to dedicated database configurations for tracking stock entries and design style matrix metrics dynamically.
- **Asynchronous Failure Queues:** Inject explicit notification steps linked directly to active administrative communication groups when runtime operations fault or trigger rate limitations.